Accelerating Deep Learning Algorithms on Memristors

Techniques for Reducing Overheads of Analog Implementation

Sparsh Mittal

Digital vs Analog implementation

- Digital circuits:
  - use standard CMOS transistors and gates
  - exclusively deal with 0 and 1

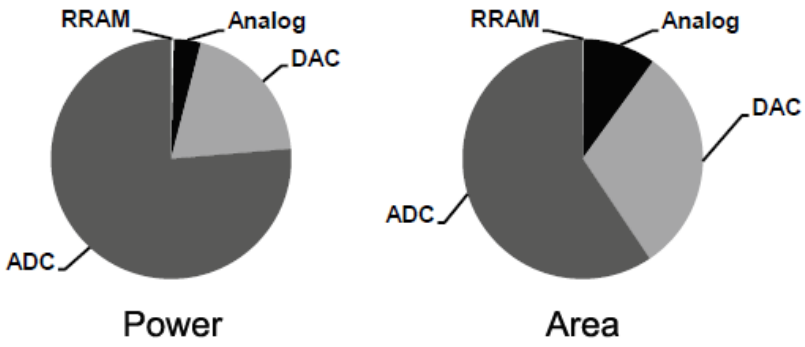
- Analog circuits:
  - use physical phenomena to carry information,
  - e.g., current in a wire or charge in a capacitor can represent a rational number
  - ReRAM allows analog implementation, which is more energy efficient than digital. Also, MVM can be parallelized significantly and need of adders is avoided.

Limitations of Analog Domain (1 of 2)

- Noise, nonlinear I-V characteristics, I/O stage resistance, process variation, change in current with temperature, etc.
- Not suitable for regular computations, e.g., financial/scientific
- Storing intermediate analog outcomes and implementing max pooling in analog domain is challenging
- Digital signal transfer allows better control and high frequency operation

Limitations of Analog Domain (2 of 2)

- Communication with digital circuitry necessitates use of ADCs/DACs, but they degrade signal precision and incur area/energy overheads
  - e.g., ADC/DAC can take 85%-98% of the total area/power of a ReRAM-based NN system



- There is (nearly) exponential relationship between the resolution (width) and cost of ADC

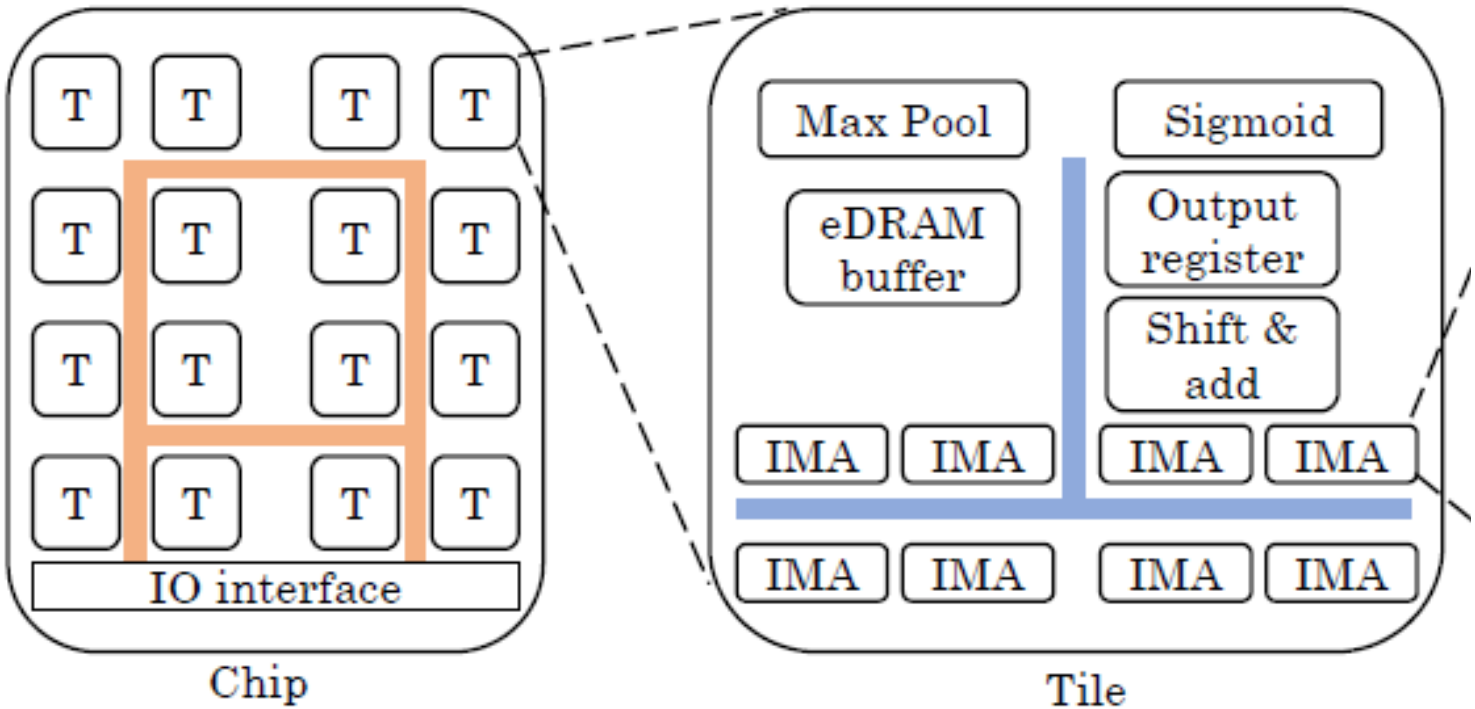
Figure from: Li et al. DAC'15

Contents

- Reducing ADC/DAC overheads [Shafiee et al.]
- Reducing port-width and area/energy overhead of eDRAM buffer [Tang et al.]
- Dealing with negative weights [Gu et al.]
- Reducing multiplications in training by ignoring multiplications of small-value [Li et al.]
- Avoiding DAC by using quantization [Xia et al.]
- Using a digital-only design [Ni et al.]
- Approximate computing for reducing requirement of precise tuning of memristors [Yantir et al.]

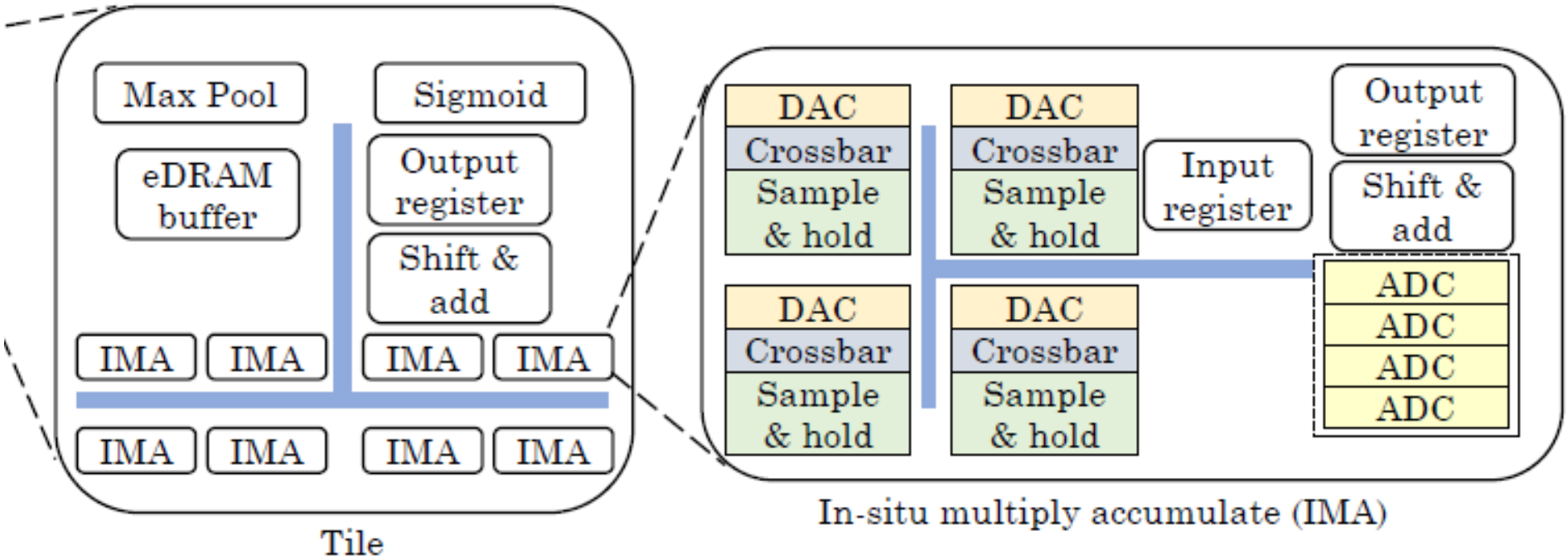
Analog architecture of Shafiee et al.

- Hierarchical topology enables high internal bandwidth, reduced data movement when aggregating results, short bitlines and wordlines in crossbars, and efficient resource partitioning across layers of a CNN.



Shafiee et al. ISCA'16

Analog architecture of Shafiee et al.



In each IMA, few ADCs are shared by multiple crossbars (MCAs)
We need enough ADCs per IMA to match the read throughput of crossbars.

Techniques for reducing ADC/DAC overheads

- Share every ADC between multiple MCAs
- When weights in a column are collectively large, store the weights in inverted form, i.e., a w-bit weight K is stored as 2^w-1-K.
  - This guarantees that MSB of the sum-of-products will be 0, which reduces ADC size by one bit
  - Provides large benefit since ADC overhead increases exponentially with resolution and ADC power is high

Shafiee et al. ISCA'16

Solved question

- Assume that the maximum value of actual weight can be (2^16- 1) (=65535),
- what will be the representation of weight 50,000 in their technique? .
- Answer:
- 65535-K = 65535-50000 = 15,535

Performing iterative multiplication (1 of 3)

- Assume we use 16-bit FxP
- Challenge: performing 16b multiplication in every memristor cell requires
  - a 16-bit DAC to provide input voltage
  - 2^{16} resistance levels in each cell, and
  - an ADC capable of handling >16 bits.
- This naïve approach has enormous overheads and is highly error-prone.
- Idea: Compute product of 16b numbers through 16 sequential operations which requires only 1-bit DAC

Performing iterative multiplication (2 of 3)

- Input is provided as multiple sequential bits. Instead of a single voltage level that represents a 16b FxP number, they provide 16 voltage levels sequentially
- Voltage level i is a 0/1 binary input representing bit i of the 16b input number
- In cycle 1: multiply-and-add bit 1 of inputs with synaptic weights and store result in an output register (after sending the sum of products through ADC).
- In Cycle 2: multiply bit 2 of inputs with synaptic weights, shift result to the left once, and add it to output register.
- Repeat this for 16 cycles to handle 16 bits.

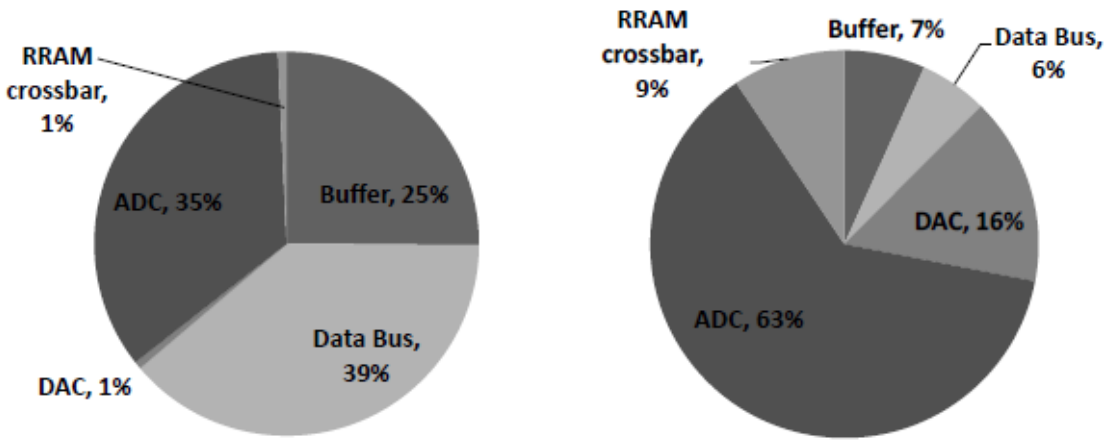
Shafiee et al. ISCA'16

Performing iterative multiplication (3 of 3)

- This algorithm is similar to the classic multiplication algorithm used in modern multipliers.
- Here, we converted input into 1b binary voltage levels, so, we performed 16 sequential operations and needed a simple 1b DAC.
- We could also accept a v-bit input voltage, which would require 16/v sequential operations and a v-bit DAC.
- Results: v = 1 is optimal and eliminates the need for an explicit DAC circuit.

Analysis of ISAAC (Shafiee et al.)

- High area cost in eDRAM buffer is caused by wide read/write ports for allowing concurrent accesses
- Shared data bus is area inefficient.
- ADC and DAC have high power overhead



(a) Area (b) Power

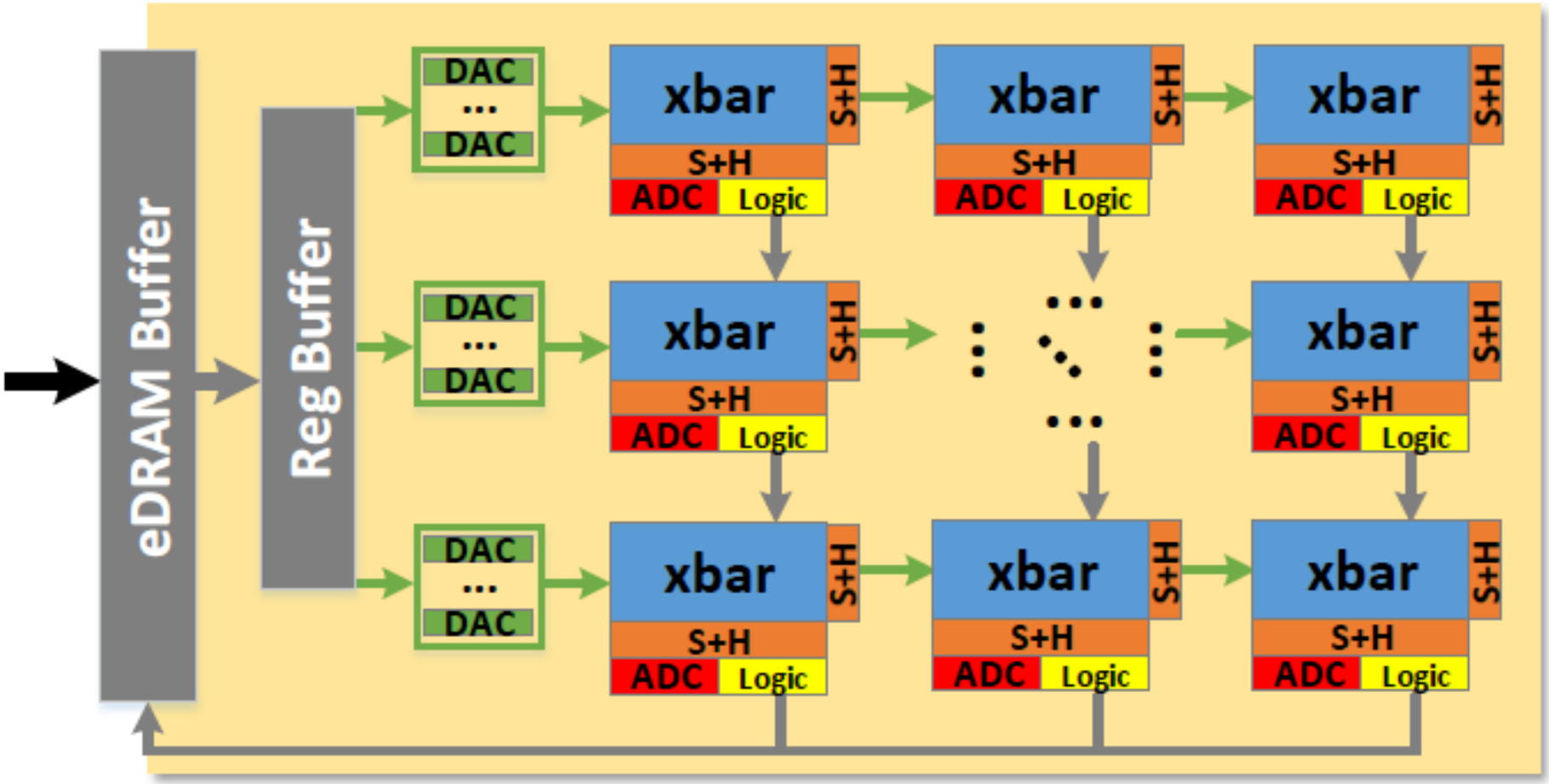
Area and power of design of ISAAC

Relation between area and port width for 64KB eDRAM buffer

Assuming 32nm technology node

bits	512	256	128	64	32	16	8
Area(mm^2)	0.41	0.28	0.16	0.14	0.11	0.09	0.09

Architecture of Tang et al. (named: AEPE)



One S+H holds temporal analog data for ADC conversion
Another holds reuse data across horizontal PEs

Tang et al. NVMSA'17

Technique of Tang et al. (named: AEPE)

Architecture parameters		
	AEPE	ISAAC
Technology @ Frequency	32nm @ 1GHz	
The number of tiles	16 × 16	168
The number of crossbars in a tile	8 × 8	96
Size of eDRAM buffer	64 KB	64KB
The number of banks	1	4
Port width (bits)	16	256
Size of register buffer	2KB	-
RRAM crossbar-based PE paramters		
Size of RRAM crossbar	128 × 128	
RRAM crossbar latency	100ns	
Resolution of the ADC, DAC, RRAM cell	9, 3, 3	11, 2, 2

Reducing ADC overheads

In ReRAM based NCS, relationship b/w resolution of ADC and DAC is

$$R_{ADC} = R_{DAC} + R_{ReRAMcell} + \log_2(Rows).$$

- Rows: number of ReRAM rows
- $R_{ReRAMcell}$ is resolution of a ReRAM cell
- Clearly, resolution of ADC needs to be high which exacerbates ADC power consumption
- Idea: DNNs can tolerate narrow values of weights/inputs/outputs => truncate weights and inputs
- Reduces ADC resolution
- Tradeoff between precision and energy efficiency

Tang et al. NVMSA'17

Reducing port width of eDRAM buffer

- Port width of eDRAM buffer is reduced by exploiting three types of data reuses:

1. Reused inputs in register buffer are not loaded repetitively from eDRAM buffer.
2. DAC converts digital inputs to analog signals that are reused across horizontal PEs.
3. ReRAM crossbar performs MAC operations and the partial sums of the convolution outputs are reused across PEs vertically.

PE = processing engine

Tang et al. NVMSA'17

Dealing with negative weights

- Store positive and negative weights in different MCAs
  - require a subtract operation for every output
- Input voltage vectors of positive MCA and negative RRAM MCA should be V_i and $-V_i$, respectively. Then,

$$\begin{aligned} \boldsymbol{V}_o &= \boldsymbol{C}^+ \cdot \boldsymbol{V}_i + \boldsymbol{C}^- \cdot (-\boldsymbol{V}_i) \\ &= (\boldsymbol{C}^+ - \boldsymbol{C}^-) \cdot \boldsymbol{V}_i = \boldsymbol{C} \cdot \boldsymbol{V}_i, \end{aligned}$$

C^+ and C^- are matrices represented by the positive and the negative MCAs

- Some designs use only non-negative weights

Gu et al. ASPDAC'15

Accelerating training

- Challenge: In "stochastic gradient descent algorithm" used for NN training, many multiplications are performed.
- They are difficult to realize in analog domain
- Ideas: approximate the error computations by ignoring values with small magnitude, such that the impact on the overall result is minimal

- Adjust convergence rate and if it falls below a threshold, stop the training process

Li et al. ASP-DAC'14

Avoiding DAC by using quantization

- Observation: In different CONV layers of a CNN, on normalizing the output values by the maximum value in that layer:
  - most values become zero, most of the remaining values are small and only a small fraction of values are large
- Idea: Normalize the data and then quantize it to 1-bit value
  - The threshold for quantization is searched for each layer separately using a greedy algorithm
- Quantized data is directly used as the analog input for MCA => need of DAC is avoided

MCA = memristor crossbar array

Xia et al. DAC'16

Performing quantization before max pooling

- Observation: Quantization after max pooling is equivalent to quantization before pooling using same threshold
- Idea: Perform quantization before pooling
- This reduces max pooling to an OR function of 1-bit inputs.
- Consider an example. Let threshold = 20

21	30
40	10

$\xrightarrow{\text{Max pooling}}$ 40 $\xrightarrow[>20?]{\text{Threshold}}$ 1

21	30
40	10

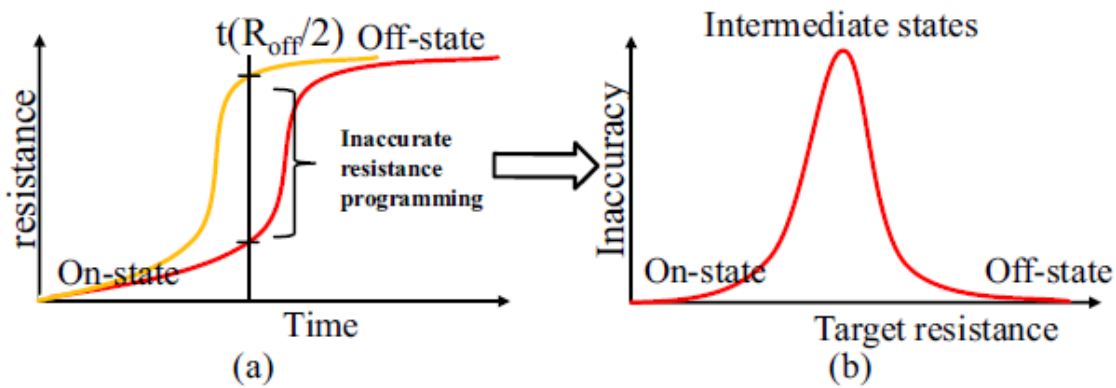
$\xrightarrow[>20?]{\text{Threshold}}$

1	1
1	0

$\xrightarrow{\text{Max pooling}}$ 1

A Digital-only Design (1 of 2)

- Challenge: On using analog values, error rate of intermediate states is much higher than that of terminal (ON/OFF) states



(a) Switching curve of RRAM under variations
(b) Programing inaccuracy for different RRAM target resistances

- Idea: Remove unreliable intermediate values by using a completely digital ReRAM crossbar design, where only binary values are used
- ADC/DAC are not required!

Ni et al. ISLPED'17

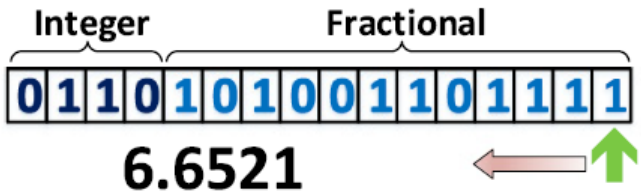
A Digital-only Design (2 of 2)

- Working: Digitize analog value by comparing the column output voltage with the reference threshold at sense amplifier.

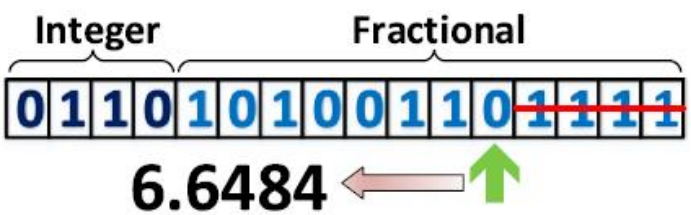
- To reduce the information loss due to digitization, increase the quantization level by using multiple thresholds
- Select the thresholds carefully to preserve most information

Approximate Computing

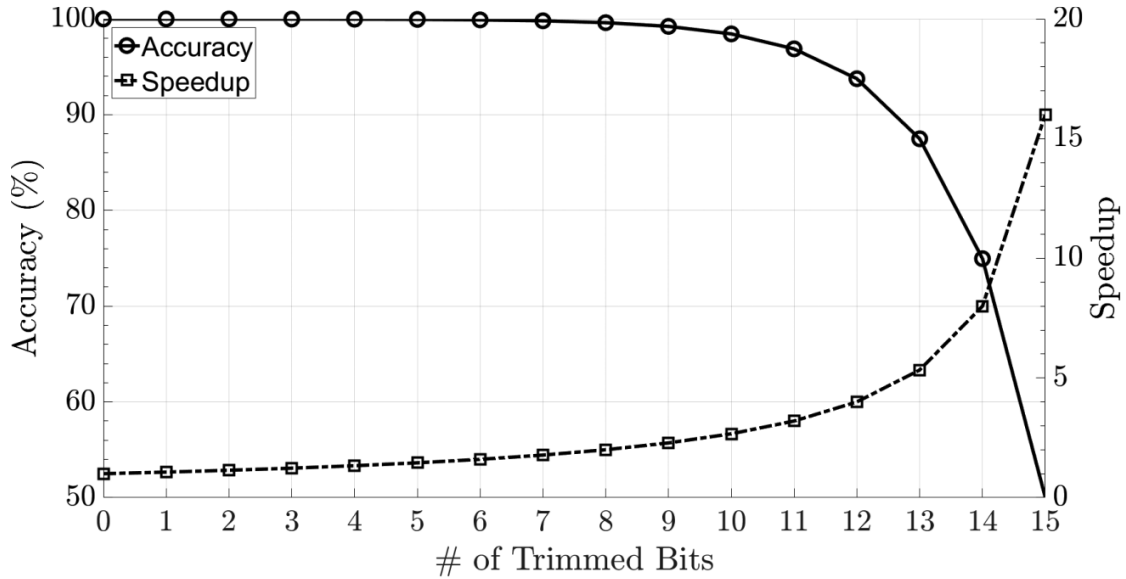
Bit trimming



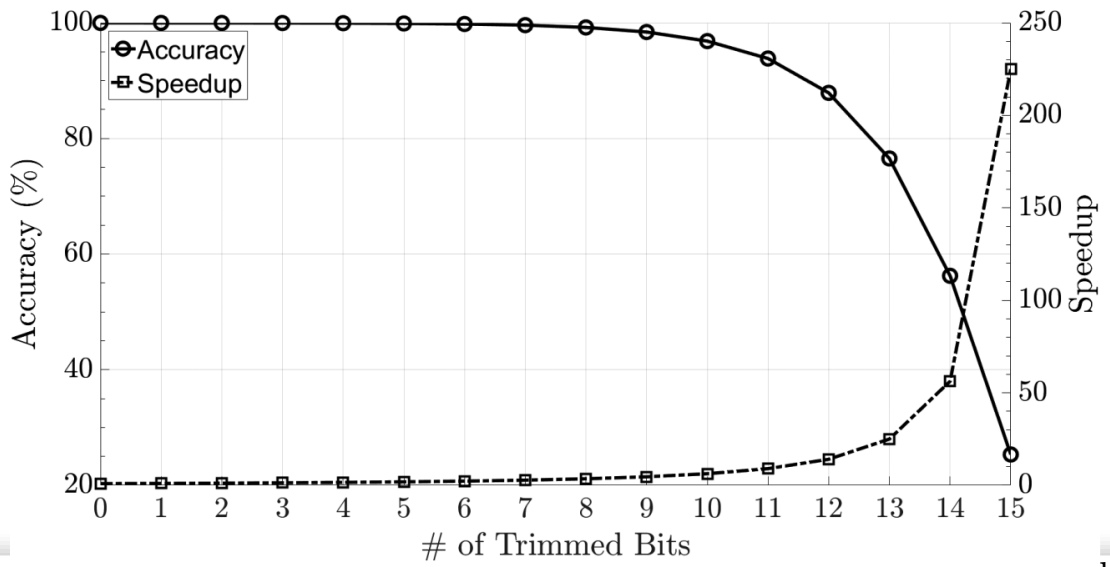
(a) Full value



(b) Approximate value



(a) Addition

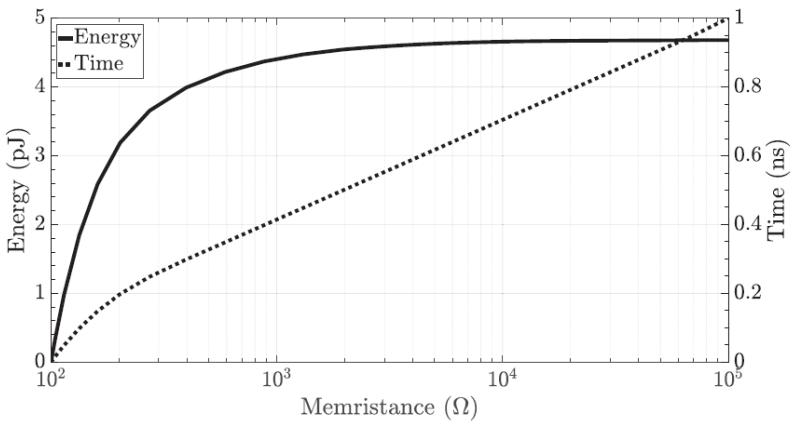


(b) Multiplication

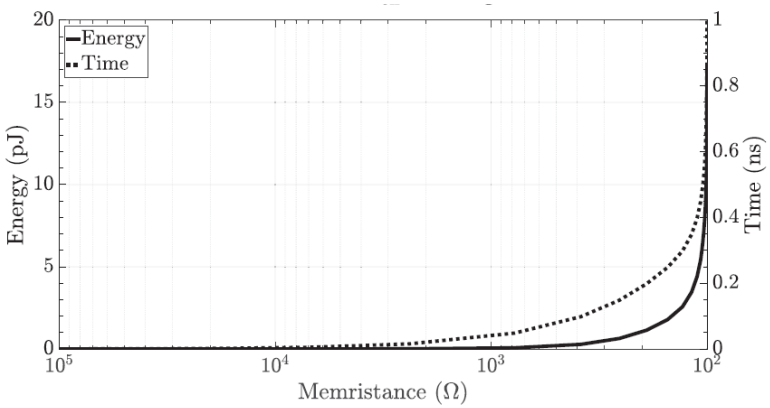
kee

Scaling resistance of memristor (1 of 2)

- Challenge: Tuning a memristor precisely to logic-0 (R_{off} =100 K ohm resistance) and logic-1 (R_{on} =100 ohm resistance) incurs large overhead
- e.g., as memristance approaches R_{on}, energy consumed increases sharply whereas the rate of change in memristance is reduced



(a) R_{on} to R_{off} switching



(b) R_{off} to R_{on} switching

Scaling resistance of memristor (2 of 2)

- Idea: Tune the memristor in a sub-range of the maximum and minimum memristance values
- This is achieved by scaling both write latency and write voltage, which also reduces write energy

Case	R_{on} (Ω)	R_{off} (Ω)	T_{write}	E_{write}	Error
Full	100	100 k	2 ns	21.7 pJ	No
Normal	1.7 k	79 k	1 ns	349.6 fJ	No
Approximate	3.2 k	40 k	0.5 ns	121.8 fJ	Yes

- "Normal" is case when effect of variation is not there.
- Limitation: On scaling further, any variability can lead to wrong value being read/written.
- Solution: do more aggressive scaling in LSB than in MSB. This is "approximate" solution mentioned in table.

Yantir et al. TECS'17

Approximate addition (1 of 2)

- Challenge: Latency of 2N-bit addition is 26N cycles
- Idea: Perform approximate addition

Result of one-bit addition

Input			Output	
A	B	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Observation: Sum is complement of C_{out} for 6 out of 8 cases

- Idea: Evaluate C_{out} precisely and then approximate Sum
- Latency reduced to 4N + 1 cycles

Approximate addition (1 of 2)

- Impact of approximation in different number of bits

APPROXIMATION IN 16-BIT ADDITION

Approximated Bits	4	8	12	14	16
Error (%)	0.006	0.098	1.56	6.25	25
Energy (pJ)	3.52	2.41	1.3	0.75	0.197
Latency (ns)	182	133	84.7	60.5	36.3

Imani et al. ISLPED'17